Pedantically Reinforced Large Language Model (PRLLM)

Walter J. Troiani Vargas [1 2] **Yan LeCun Yoshua Bengio**

Abstract

This work offers a **practical tutorial on language model (LM) preference alignment** using Reinforcement Learning from Human Feedback **(RLHF)**. We compare three prominent algorithms in this domain: PPO, DPO and GRPO. Leveraging the high-level TRL (Hugging Face, 2024) and Transformers libraries by HuggingFace, the paper aims to not only demystify the implementation process for novice AI engineers and NLP practitioners, addressing a lack of accessible, high-quality learning resources, but also to establish a practical comparative **baseline between algorithms** from a more humble computational power viewpoint, clarifying generalist claims made in the literature. We present a practical case study where we fine-tune SmolLM2 to produce **Pedantic-SmolLM2** in its 3 variants. This process enforces a pedantic, intellectual, eloquent expression. Both DPO and GRPO demonstrated to be the most resource efficient alternatives in terms of time, simplicity and data hungriness.

1. Introduction

Large language models (LLMs) have recently taken the world and AI field by storm. The recency (¡3 years) of this kind of models creates a big difficulty for the non-expert to not only find high quality resources, but to fact-check the validity of the aforementioned, often without available source code, model, data sets and randomness seeds to make it reproducible.

This is exactly what this publication aims to solve by presenting not only a theoretical introduction but also, most importantly, open source code and data sets through both HuggingFace and Github. By making use of the high level interfaces and algorithmical implementations provided by the HuggingFace libraries (Wolf et al., 2020) (transformers, trl, data sets), we also present a real world scalable solution to the preference alignment problem in LLMs without having to implement any procedure from scratch and avoiding code liability and reduce barrier to entrance.

The end goal of this work is to align a language model to the author preferences, making it more sophisticated and pedantic. Our study case revolves around the tiny SmolLM2 (Allal et al., 2025)in its 135 million parameter version, developed and sourced by HuggingFace. Three preference alignment algorithms will be employed and compared to achieve this goal, related to the reinforcement learning domain, namely: Proximal Policy Optimization (PPO) (Schulman et al., 2017), Direct Policy Optimization (DPO) (Rafailov et al., 2023) (Hejna et al., 2023), and Group Relative Policy Optimization (GRPO) (Ramesh et al., 2024).

2. Problem

2.1. Context

The training of transformer-based language models is typically decomposed into three distinct phases, which collectively transform massive data sets into helpful conversational agents:

- **Pretraining:** An initial transformer model $\mathcal{M}$, with a tokenized vocabulary of size V, is trained on a massive corpus $\mathcal{C}$ of high-quality raw text. The objective is to minimize the cross-entropy loss between the model's token predictions and the actual tokens in the corpus, treating the vocabulary as a categorical distribution conditioned on preceding context. Training typically occurs at the scale of billions to trillions of tokens.
- **Supervised Fine-Tuning (SFT):** Following pretraining, the model undergoes fine-tuning on a smaller but substantial data set of curated question-answer pairs $\mathcal{P} = \{(Q_i, A_i)\}_{i=1}^{N}$. This phase employs a similar cross-entropy objective but encourages the model to adopt conversational patterns and stylistic conventions characteristic of helpful chatbots, rather than maintaining general language modeling capabilities.
- **Reinforcement Learning Alignment:** The final stage further aligns the model with human preferences (Kaufmann et al., 2024) using reinforcement learning algorithms often trained with both positive and negative examples $\mathcal{T} = \{(Q_i, A_i, R_i)\}_{i=1}^{N}$, typically optimizing reward functions that capture desired conversational qualities while maintaining coherence and preventing

regression from SFT capabilities.

2.2. Motivation

Now that the context has been laid out, let's introduce the motivation. Inspired by sophisticated art and high-society intellectual entertainment (burialgoods, 2024), we want to elevate our model's linguistic sophistication and vocabulary. Our core hypothesis is that RLHF can achieve this more effectively than SFT, being significantly less data-hungry and offering better generalization.

While our algorithm selection has some arbitrary and popularity-driven aspects, we'll employ the three aforementioned RL methods to fine-tune the already pretrained and instruction-tuned weights of SmolLM2. This serves both to advance toward our goal and as a comprehensive learning experience in model alignment, but details on implementations are out of the scope of this work.

Since the model outputs natural language text, assessing pedantic sophistication requires either LLM and human judgment. We're using Qwen2.5-0.5B for LLM-as-a-judge evaluation (Li et al., 2024) to avoid the infamous narcissistic bias (Liu et al., 2024) , though the model's tiny size will likely impair evaluation quality, making human assessment absolutely necessary.

Note that our baseline is obviously the base SmolLM2-135M model, making comparative evaluation straightforward and crucial for measuring improvements or detecting deterioration from issues like catastrophic forgetting or reward hacking (Skalse et al., 2025).

3. Data Processing

Although this project can seem to be of an algorithmic nature, the most difficult parts are actually data preparation and curation and finally hyperparameter tuning given the TRL library implementations. Given the humongous nature of LLMs and the limited computational resources, simple tuning by hand will be employed. So our goal is to generate and curate data well enough but avoiding doing all the work manually.

- **Initial Prompt Database** $\mathcal{C}$: This will simply be created by ingestion of already existing prompt data sets in Huggingface, limiting it to only 1500 entries.
- **Question-Approved-Rejected Triples** $\mathcal{T}$: This data set with responses will be generated using the baseline SmolLM2 to avoid manual labor but also to avoid drastically changing the distribution of the model and using out-of-vocabulary (OOV) tokens. Given that the execution led to some NA responses and some really poor quality examples, ChatGPT 4.0 will be used to fill the gap in these approximately 100 examples without response. This inevitably entails some potential risks as previously mentioned, but given the authors limited time window, this will be the best semi-automated option.
- **Question-Answer Pairs** $\mathcal{D}$: To obtain such a data set to perform SFT, manual generation by language experts would be highly relevant, however, the accepted/rejected pairs of $\mathcal{T}$ could also be used by keeping only the accepted responses, although the responses in many cases could be non ideal.

The size of 1500 examples (150 test, 150 validation, 1200 train) has been hand-tuned to adjust for the computational power of commodity computers and small training times ($\leq 4h$), so it could become a possible bottleneck in the experimentation.

4. Experiments & Discussion

All experiments were conducted using a fixed random seed of 42 to ensure reproducibility across runs. Training and evaluation were performed on *Google Colab T4* instances equipped with 12 GB of system RAM, 15 GB of GPU VRAM, and approximately 100 GB of temporary disk storage. The NVIDIA T4 GPU offers peak performance near 8 TFLOPs in FP32 precision, which, while sufficient for small- to medium-scale transformer fine-tuning, imposes strict constraints on model size, batch scheduling, and experiment duration. All fine-tunings are produced in the entirety of the model and not on LoRA adapters as is commonly done.

4.1. Reward Model Training and PPO

Firstly, the reward model, trained over 300 optimization steps (20h on CPU only, 1h on T4 GPU), exhibited a clear and consistent convergence trend as illustrated in Figure 1. The training loss decreased sharply from an initial value of approximately 0.78 to below 0.05 by the end of training, so it appears as the to discriminate between more and less desirable completions.

Quantitative evaluation after two epochs yielded an **evaluation accuracy of 94.58%** and a loss of 0.1911. The **mean reward of** -4.07 and margin of 5.80 indicate that while the model maintains strong relative separation between preferred and dispreferred samples, the overall reward scale remains centered below zero. The maximum reward reached 2.28, while the minimum reward was -9.08.

Applying this reward model in a full PPO pipeline yields the results shown, with this being the best of five runs using different hyperparameters to stabilize convergence, maximize reward, and minimize distribution divergence. From

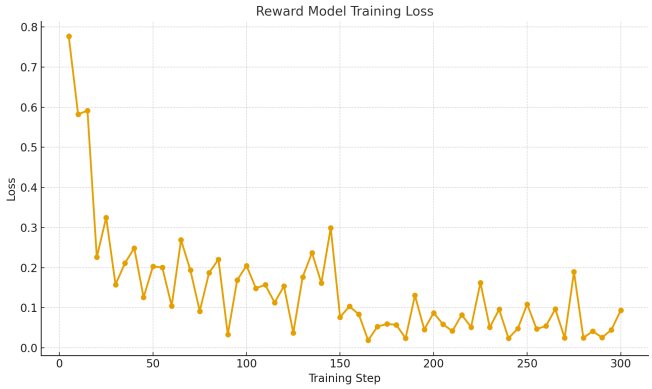

Figure 1. Reward model training loss. The model shows consistent training convergence with loss stabilizing below 0.1.

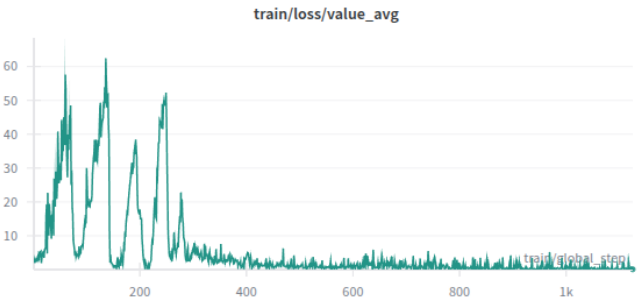

Figure 2. PPO Clipped policy gradient training loss over training steps.

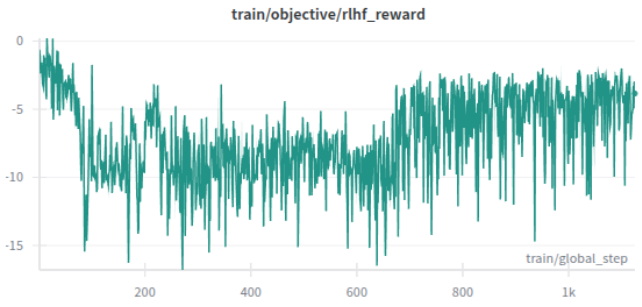

Figure 3. PPO Rewards over training steps.

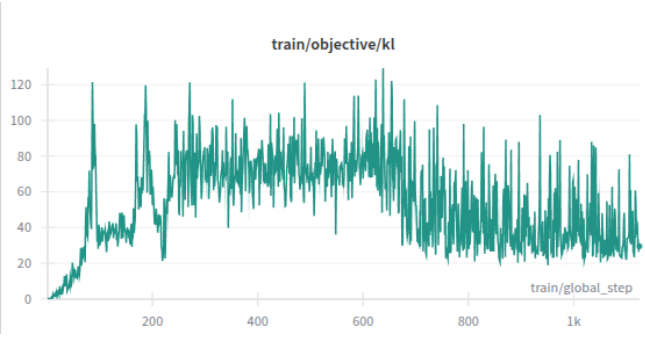

Figure 4. PPO Kullback-Leiber divergence over training steps.

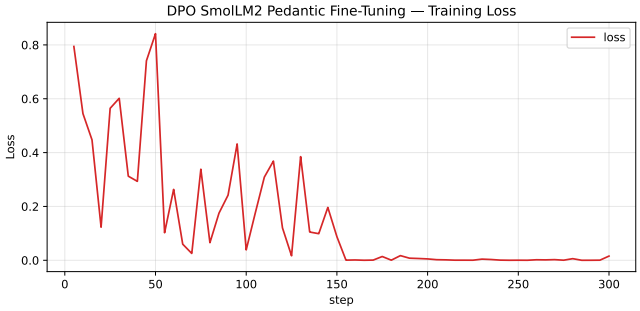

Figure 5. DPO Preference logistic loss over training steps.

Figure [4], the model has clearly diverged drastically from its original distribution. The loss (Figure [2]) exhibits unusual patterns, and the average reward appears to converge around -3 (Figure [3]). Manual inspection of the outputs reveals that the model frequently produces a lot of filler words such as ”or,” ”the,” and ”and,” with almost no coherent text, indicating both reward hacking and catastrophic forgetting, double the fun, for the same price. This behavior arises because the reward model fails to capture true pedanticity, while the LLM exploits spurious patterns imposed by the reward model. The reward hacking then triggers catastrophic forgetting, as controlling divergence becomes essentially impossible even for huge values of $\beta = 200$.

The main takeaway is that PPO depends entirely on the reward model which becomes the bottleneck, which in this case seems to be a lack of training data and/or epochs.

4.2. DPO

Direct Policy Optimization seeks to minimize the log likelihood directly seeing ”preferred” and ”other” pair of responses, aligning more with SFT than to PPO. Looking closely at the loss [5] we can clearly see that the model has probably overfitted the training set in a value near 0. This is not an alarming scenario in the fine-tuning practice, specially given that the rewards [6] seem to have converged to positive for ”chosen” responses and negative in the case of ”rejected” responses.

In this training, KL divergence (Shlens, 2014) was unfortunately not recorded, but inference results suggest that the model has diverged slightly, though not excessively, and that

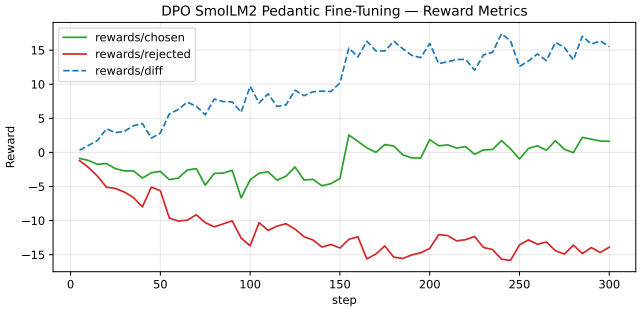

Figure 6. DPO rewards over training steps (Smoothed).

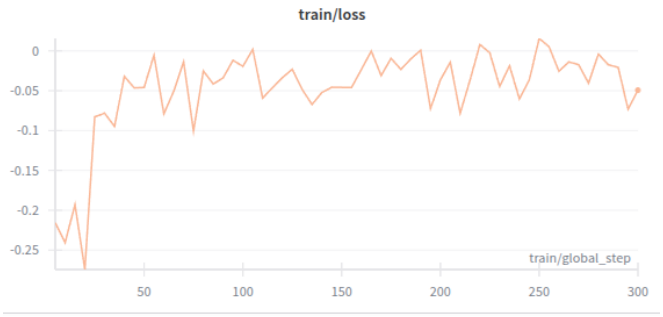

Figure 7. GRPO Reward-Weighted loss over training steps.

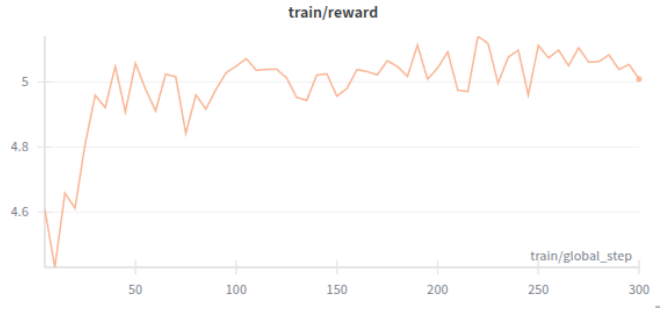

Figure 8. GRPO weighted average reward over training steps.

the generated text exhibits a more pedantic and rude style, sometimes failing to answer the question directly, instead prompting philosophical reflections or abruptly changing the topic.

4.3. GRPO

Group Relative Policy Optimization similarly to DPO does not depend on an explicit reward model but rather on programatic reward functions (DeepSeek-AI et al., 2025), which give extreme versatility on the cost of a more feature engineering intense task. In our case its crucial to engineer with these functions what characterizes a pedantic text, which can be simplified to:

- **Lexical complexity (w=0.18):** rewards longer words, higher type-token ratio, and rarer vocabulary.
- **Length (w=0.12):** favors moderate-length completions to avoid trivially short answers.
- **Syntactic complexity (w=0.18):** rewards longer sentences, deeper parse trees, subordinate clauses, and passive constructions.
- **Stylistic precision (w=0.15):** rewards precision adverbs, low contraction usage, and higher readability grade (Flesch-Kincaid).
- **Concreteness and specificity (w=0.12):** rewards presence of concrete references such as numbers, dates, and proper nouns.
- **Keyword density (w=0.15):** rewards the use of academic or technical keywords.
- **Hedge penalty (w=0.10):** penalizes vague or hedging phrases to promote precise statements.

Collectively, these functions produce a weighted score (See weights above) trying to reflect the degree of pedantry in a text. Note that the keyword lists are pretty basic and not exhaustive. Looking closely at the loss function during training [7] we can see that has somewhat high variance but tries to stabilize near the origin, and judging from the KL divergence [9], the model has not diverged greatly from the original distribution, which is a great sign. Finally if we take a look at the rewards over time 8, the model has tuned its parameters to get an average 0.5 higher reward successfully. Judging from its inference we can see that this is the most stable and less divergent model while also up leveling the language a bit. With more data, this algorithm seems that it would scale the best among its competitors given the observed stability.

4.4. Extra: SFT

Altough the following approach is certainly no reinforcement learning algorithm, the authors were curious to see the relationship between SFT and DPO, given that we could use the ”chosen” responses, although the algorithms and data needs are completely different. A small sample size of 1200 won’t probably be enough to satisfiy the hunger of data of SFT but nevertheless, with all the already laid out pipeline its easy to test with the TRL library how a full model finetuning would result. Given we return to a

Figure 9. GRPO Kullback-Leiber divergence over training steps.

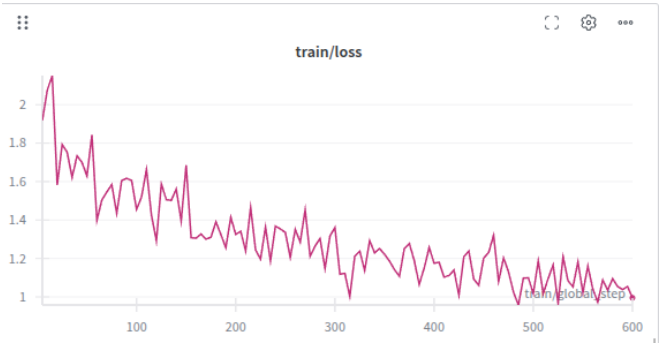

Figure 10. SFT Cross-Entropy loss over training steps.

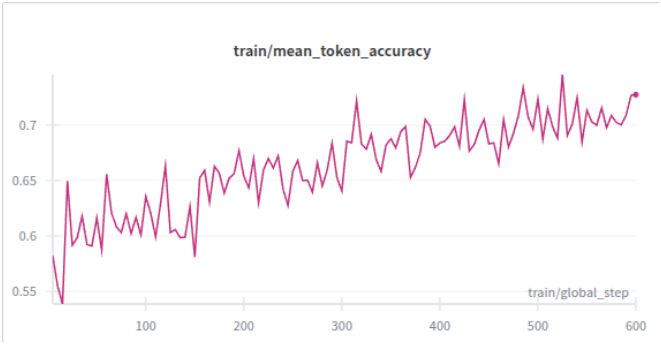

Figure 11. SFT Mean token accuracy during training.

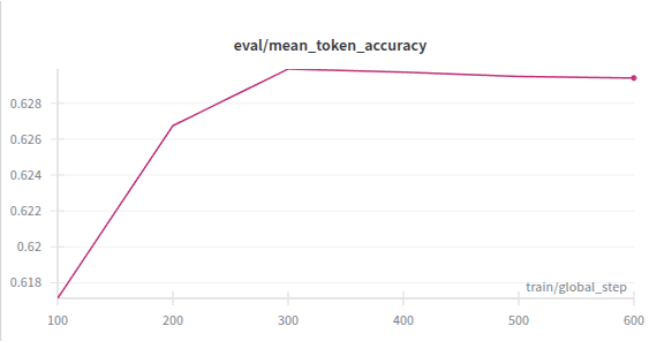

Figure 12. SFT Mean token accuracy during evaluation.

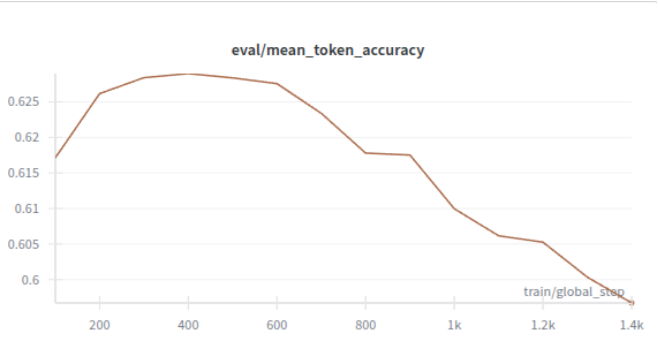

Figure 13. SFT Mean token accuracy during evaluation (Overfitting example)

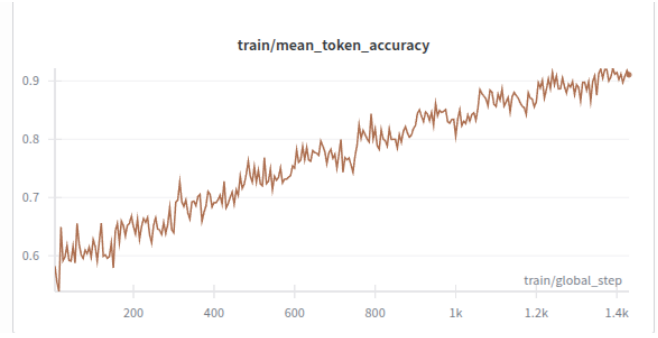

Figure 14. SFT Mean token accuracy during training (Overfitting example)

supervised setting, the model will be evaluated on the token prediction proxy task. As seen on the decreasing loss function [10], increasing training token accuracy [11] and plateauing evaluation token accuracy [12], it can inferred that the model has learned some patterns from the training data.

Similar to past model trainings, three executions were performed but only this one was kept due to a lack of overfitting, in contrast to the rest [13]. Likely, the long plateau indicates a lack of volume and variety given that it does not improve on unseen data despite having a rising training token accuracy [14].

4.5. Evaluation Results

The evaluation of these 4 models and the baseline has been performed using both human evaluation and LLM-as-judge

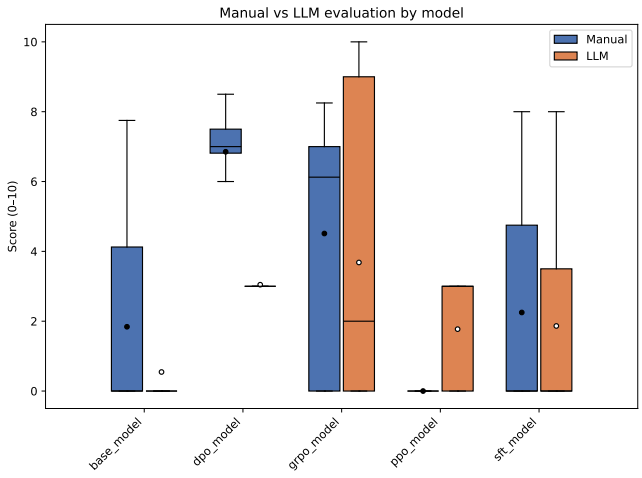

Figure 15. Evaluation of the 4 tuned models against the baseline using manual and LLM-as-Judge evaluation.

in a set of 22 hand-crafted questions, using a **pedanticity score** from 0-10 to asses the style of the question; note that the helpfulness of the aforementioned contrary to popular LLM training is not taken into account in our case, although its a nice to have. It must be said before any analysis that many times the models have returned empty string responses, specially Base, DPO and PPO.

The first observation that can infered from [15] is that this LLM evaluation on this test sample is not realiable due to the lack high bias it has on some models (DPO, Base) or high variance (GRPO) and it hasn't been able to detect the nonsensical output text of the PPO model. The rating is also pessimistic on the 2 only performant models (DPO, GRPO), therefore this evaluation won't be taken into account. One of the hypotheses for such failure is probably the reduced size of this quantized LLM.

Now paying attention to the subjective human evaluation which focuses on the pedantic style, we can clearly see that DPO and GRPO are the clear winners, DPO being a bit more pedantic on average and with less variability. However, taking a closer look at both model responses it can be seen that DPO although it becomes more pedantic than DPO it becomes crazier (starts to state and ask non sensical statements) while GRPO increases a bit the level of its vocabulary while remaining a helpful assitant, so depending in the view point one could argument GRPO is the best model so far given its stability after fine tuning.

Another important observation to make is that while SFT is marginally better than the baseline model, its data efficiency is really low compared to DPO but its more stable.

5. Conclusions & Future Work

The results of this work can be found self-containedly in the following GitHub repository (eZWALT, b), and the data or models can be found on the corresponding Hugging Face profile (eZWALT, a). Any suggestions are welcome in the form of merge requests.

GRPO and DPO have proven to be simpler yet effective algorithms for performing preference alignment without the daunting task of training a full reward model, as required by PPO. However, it is worth noting the significant trade-off being made here: simplicity and agility (quicker engineering for MVPs) versus quality and scalability. For most end users who are not foundational model research organizations, the authors recommend GRPO when the alignment goal is easily characterizable, and DPO otherwise.

Although the attempt to obtain a more pedantic model was not fully achieved to the initially expected degree (burialgoods, 2024), this work has provided both the authors and readers with a solid and broad understanding of preference alignment and language model fine-tuning. Future work that could improve the quality of results includes expanding the dataset size and performing prior SFT or additional pretraining using transcriptions from (burialgoods, 2024).

For subsequent iterations of this project, more variations of RLHF algorithms should be explored to better understand trade-offs, with special attention devoted to making PPO work properly. Nevertheless, it does not seem that hyperparameter tuning alone will solve the core issues related to reward modeling,volume and variety of data likely will. Additionally, for PPO, applying LoRA could be beneficial, as freezing most weights may help mitigate catastrophic forgetting and reward hacking to some extent. Similarly, SFT would also greatly benefit from scaling both the volume and diversity of data.

The reward dataset's null values after generation were filled using GPT-4, which has a very distinct vocabulary and structure compared to SmolLM2, likely affecting all subsequent training runs. This should be avoided in future iterations. Regarding evaluation, several improvements could enhance statistical accuracy, namely:

- Using a much larger LLM to perform LLM-as-Judge evaluation.
- Expanding and leveraging the functions defined in GRPO, as these capture some characteristics of pedantic text.
- Utilizing the complete evaluation dataset (150 examples) to improve distributional accuracy.
- Implementing a retry mechanism to avoid empty-string responses.

Overall, this work has pushed the limits of what a commodity computer can achieve in preference alignment. It can be argued that with more compute time, higher-quality training remains possible within this small parameter range (<1B) without requiring multiple machines or GPUs.

A. PPO

Proximal Policy Optimization (PPO) is a policy-gradient method that maximizes expected cumulative reward while constraining each policy update to stay close to a reference policy, typically the previous policy, to avoid large destructive updates. Using a policy π_θ and advantages $\hat{A}_t$, PPO optimizes a clipped surrogate objective,

$$\mathcal{L}_{\text{PPO}}(\theta) = E_t\Big[\min\Big(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\Big)\Big], \tag{1}$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ is the probability ratio and ϵ is a small clipping hyperparameter. In RLHF deployments a KL penalty or explicit KL constraint with a reference policy π_{ref} is often added to prevent distributional drift:

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{PPO}}(\theta) - \beta\,\text{KL}\big(\pi_\theta \,\|\, \pi_{\text{ref}}\big). \tag{2}$$

B. DPO

Direct Preference Optimization (DPO) is a preference-based fine-tuning method that directly trains a conditional language model to prefer human-preferred outputs over alternatives using a logistic (contrastive) loss, avoiding an explicit reward model or RL loop. Given a prompt x, a preferred response y^+ and a less-preferred response y^-, DPO minimizes the negative log-sigmoid of the log-probability ratio:

$$\mathcal{L}_{\text{DPO}}(\theta) = -\log\sigma\left(\log\frac{\pi_\theta(y^+ \mid x)}{\pi_\theta(y^- \mid x)}\right) = -\log\sigma\big(\log\pi_\theta(y^+ \mid x) - \log\pi_\theta(y^- \mid x)\big), \tag{3}$$

where σ is the logistic sigmoid. Intuitively, DPO encourages the model to increase probability mass on preferred replies relative to alternatives, using a supervised-style contrastive objective.

C. GRPO

Group Relative Policy Optimization (GRPO) generalizes reward optimization to incorporate programmatic or group-aware rewards and explicit divergence regularization, enabling robustness across groups or reward components. In practice GRPO can be stated as maximizing a (possibly group-robust) expected reward while penalizing divergence from a reference policy:

$$\max_\theta\ \min_{g\in\mathcal{G}}\ E_{\pi_\theta}\big[R_g(s,a)\big]\ -\ \lambda\, D\big(\pi_\theta \,\|\, \pi_{\text{ref}}\big), \tag{4}$$

where $\mathcal{G}$ indexes groups or reward channels, R_g denotes per-group reward functions (in our work these are the programmatic GRPO reward components), $D(\cdot\|\cdot)$ is a divergence (often KL), and λ trades off reward versus distribution fidelity. GRPO implementations commonly combine per-component scalar rewards (or a weighted sum) and perform policy optimization with the same practical machinery as PPO, but using these engineered reward signals.

D. SFT

Supervised Fine-Tuning (SFT) adapts a pretrained language model to paired examples (x, y) by minimizing the token-level negative log-likelihood (cross-entropy). For a target sequence $y = (y_1, \ldots, y_T)$ the SFT objective is

$$\mathcal{L}_{\text{SFT}}(\theta) = -\sum_{t=1}^{T}\log\pi_\theta(y_t \mid x, y_{<t}), \tag{5}$$

which directly trains the model to predict the next token given context. SFT is simple and stable, but it treats the human preference signal as ground-truth tokens rather than pairwise preferences, and typically requires larger curated datasets to generalize style or behavior changes.

References

Allal, L. B., Lozhkov, A., Bakouch, E., Blázquez, G. M., Penedo, G., Tunstall, L., Marafioti, A., Kydlíček, H., Lajarín, A. P., Srivastav, V., Lochner, J., Fahlgren, C., Nguyen, X.-S., Fourrier, C., Burtenshaw, B., Larcher, H., Zhao, H., Zakka, C., Morlon, M., Raffel, C., von Werra, L., and Wolf, T. Smollm2: When smol goes big – data-centric training of a small language model, 2025. URL https://arxiv.org/abs/2502.02737.

burialgoods. brother may i have some oats. YouTube video, 2024. URL http://www.youtube.com/watch?v=O7FIiYsVy3U. Accessed on November 6, 2025.

DeepSeek-AI, Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., Xu, R., Zhu, Q., Ma, S., Wang, P., Bi, X., Zhang, X., Yu, X., Wu, Y., Wu, Z. F., Gou, Z., Shao, Z., Li, Z., Gao, Z., Liu, A., Xue, B., Wang, B., Wu, B., Feng, B., Lu, C., Zhao, C., Deng, C., Zhang, C., Ruan, C., Dai, D., Chen, D., Ji, D., Li, E., Lin, F., Dai, F., Luo, F., Hao, G., Chen, G., Li, G., Zhang, H., Bao, H., Xu, H., Wang, H., Ding, H., Xin, H., Gao, H., Qu, H., Li, H., Guo, J., Li, J., Wang, J., Chen, J., Yuan, J., Qiu, J., Li, J., Cai, J. L., Ni, J., Liang, J., Chen, J., Dong, K., Hu, K., Gao, K., Guan, K., Huang, K., Yu, K., Wang, L., Zhang, L., Zhao, L., Wang, L., Zhang, L., Xu, L., Xia, L., Zhang, M., Zhang, M., Tang, M., Li, M., Wang, M., Li, M., Tian, N., Huang, P., Zhang, P., Wang, Q., Chen, Q., Du, Q., Ge, R., Zhang, R., Pan, R., Wang, R., Chen, R. J., Jin, R. L., Chen, R., Lu, S., Zhou, S., Chen, S., Ye, S., Wang, S., Yu, S., Zhou, S., Pan, S., Li, S. S., Zhou, S., Wu, S., Ye, S., Yun, T., Pei, T., Sun, T., Wang, T., Zeng, W., Zhao, W., Liu, W., Liang, W., Gao, W., Yu, W., Zhang, W., Xiao, W. L., An, W., Liu, X., Wang, X., Chen, X., Nie, X., Cheng, X., Liu, X., Xie, X., Liu, X., Yang, X., Li, X., Su, X., Lin, X., Li, X. Q., Jin, X., Shen, X., Chen, X., Sun, X., Wang, X., Song, X., Zhou, X., Wang, X., Shan, X., Li, Y. K., Wang, Y. Q., Wei, Y. X., Zhang, Y., Xu, Y., Li, Y., Zhao, Y., Sun, Y., Wang, Y., Yu, Y., Zhang, Y., Shi, Y., Xiong, Y., He, Y., Piao, Y., Wang, Y., Tan, Y., Ma, Y., Liu, Y., Guo, Y., Ou, Y., Wang, Y., Gong, Y., Zou, Y., He, Y., Xiong, Y., Luo, Y., You, Y., Liu, Y., Zhou, Y., Zhu, Y. X., Xu, Y., Huang, Y., Li, Y., Zheng, Y., Zhu, Y., Ma, Y., Tang, Y., Zha, Y., Yan, Y., Ren, Z. Z., Ren, Z., Sha, Z., Fu, Z., Xu, Z., Xie, Z., Zhang, Z., Hao, Z., Ma, Z., Yan, Z., Wu, Z., Gu, Z., Zhu, Z., Liu, Z., Li, Z., Xie, Z., Song, Z., Pan, Z., Huang, Z., Xu, Z., Zhang, Z., and Zhang, Z. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025. URL https://arxiv.org/abs/2501.12948.

eZWALT. Hugging face profile and models. Hugging Face, a. URL https://huggingface.co/eZWALT. Accessed on November 6, 2025.

eZWALT. Rl-reinforcement-learning project readme: Preference alignment tutorial. GitHub Repository, b. URL https://github.com/eZWALT/RL-Reinforcement-Learning/blob/main/PROJECT/README.md. Accessed on November 6, 2025.

Hejna, J., Gao, T., Alayrac, J.-B., Kay, J., Schwarzer, M., Hassabis, D., Vinyals, O., and Irving, G. Contrastive preference learning: Reward-free reinforcement learning from human feedback. *arXiv preprint arXiv:2305.18290*, 2023.

Hugging Face. Transformer reinforcement learning (trl) documentation. https://huggingface.co/docs/trl, 2024.

Kaufmann, T., Kreutzer, J., and Riezler, S. Reinforcement learning from human feedback: Progress and challenges. *arXiv preprint arXiv:2403.09032*, 2024.

Li, H., Dong, Q., Chen, J., Su, H., Zhou, Y., Ai, Q., Ye, Z., and Liu, Y. Llms-as-judges: A comprehensive survey on llm-based evaluation methods. *arXiv preprint arXiv:2412.05579*, 2024. URL https://arxiv.org/abs/2412.05579.

Liu, Y., Moosavi, N. S., and Lin, C. Llms as narcissistic evaluators: When ego inflates evaluation scores, 2024. URL https://arxiv.org/abs/2311.09766.

Rafailov, R., Sharma, A., Mitchell, E., and Finn, C. Direct preference optimization: Your language model is secretly a reward model. *arXiv preprint arXiv:2305.18290*, 2023.

Ramesh, A., Xu, C., Lemoine, B., Chiappa, S., et al. Group robust preference optimization. *arXiv preprint arXiv:2401.01337*, 2024.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Shlens, J. Notes on kullback-leibler divergence and likelihood, 2014. URL https://arxiv.org/abs/1404.2000.

Skalse, J., Howe, N. H. R., Krasheninnikov, D., and Krueger, D. Defining and characterizing reward hacking, 2025. URL https://arxiv.org/abs/2209.13085.

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., et al. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 38–45. Association for Computational Linguistics, 2020. URL https://arxiv.org/abs/1910.03771.